A Technical Introduction to BERT language model

Section 1

Given two sentences, the model predicts if they appear sequentially in the training corpus, outputting either [IsNext] or [NotNext]. During training, the algorithm sometimes samples two sentences from a single continuous span in the training corpus, while at other times, it samples two sentences from two discontinuous spans. The first sentence starts with a special token, [CLS] (for "classify"). The two sentences are separated by another special token, [SEP] (for "separate"). After processing the two sentences, the final vector for the [CLS] token is passed to a linear layer for binary classification into [IsNext] and [NotNext]. For example:

Section 2

In masked language modeling, 15% of tokens would be randomly selected for masked-prediction task, and the training objective was to predict the masked token given its context. In more detail, the selected token is:

Section 3

Embedding This section describes the embedding used by BERTBASE. The other one, BERTLARGE, is similar, just larger. The tokenizer of BERT is WordPiece, which is a sub-word strategy like byte-pair encoding. Its vocabulary size is 30,000, and any token not appearing in its vocabulary is replaced by [UNK] ("unknown").

Section 4

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.

Section 5

BERT is meant as a general pretrained model for various applications in natural language processing. That is, after pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as natural language inference and text classification, and sequence-to-sequence-based language generation tasks such as question answering and conversational response generation. The original BERT paper published results demonstrating that a small amount of finetuning (for BERTLARGE, 1 hour on 1 Cloud TPU) allowed it to achieved state-of-the-art performance on a number of natural language understanding tasks:

Section 6

replaced with a [MASK] token with probability 80%, replaced with a random word token with probability 10%, not replaced with probability 10%. The reason not all selected tokens are masked is to avoid the dataset shift problem. The dataset shift problem arises when the distribution of inputs seen during training differs significantly from the distribution encountered during inference. A trained BERT model might be applied to word representation (like Word2Vec), where it would be run over sentences not containing any [MASK] tokens. It is later found that more diverse training objectives are generally better. As an illustrative example, consider the sentence "my dog is cute". It would first be divided into tokens like "my1 dog2 is3 cute4". Then a random token in the sentence would be picked. Let it be the 4th one "cute4". Next, there would be three possibilities:

Section 7

Architectural family The encoder stack of BERT has 2 free parameters: L {\displaystyle L} , the number of layers, and H {\displaystyle H} , the hidden size. There are always H / 64 {\displaystyle H/64} self-attention heads, and the feed-forward/filter size is always 4 H {\displaystyle 4H} . By varying these two numbers, one obtains an entire family of BERT models. For BERT:

Section 8

GLUE (General Language Understanding Evaluation) task set (consisting of 9 tasks); SQuAD (Stanford Question Answering Dataset) v1.1 and v2.0; SWAG (Situations With Adversarial Generations). In the original paper, all parameters of BERT are fine-tuned, and recommended that, for downstream applications that are text classifications, the output token at the [CLS] input token is fed into a linear-softmax layer to produce the label outputs. The original code base defined the final linear layer as a "pooler layer", in analogy with global pooling in computer vision, even though it simply discards all output tokens except the one corresponding to [CLS] .

Section 9

The three attention matrices are added together element-wise, then passed through a softmax layer and multiplied by a projection matrix. Absolute position encoding is included in the final self-attention layer as additional input.

Section 10

Token type: The token type is a standard embedding layer, translating a one-hot vector into a dense vector based on its token type. Position: The position embeddings are based on a token's position in the sequence. BERT uses absolute position embeddings, where each position in a sequence is mapped to a real-valued vector. Each dimension of the vector consists of a sinusoidal function that takes the position in the sequence as input. Segment type: Using a vocabulary of just 0 or 1, this embedding layer produces a dense vector based on whether the token belongs to the first or second text segment in that input. In other words, type-1 tokens are all tokens that appear after the [SEP] special token. All prior tokens are type-0. The three embedding vectors are added together representing the initial token representation as a function of these three pieces of information. After embedding, the vector representation is normalized using a LayerNorm operation, outputting a 768-dimensional vector for each input token. After this, the representation vectors are passed forward through 12 Transformer encoder blocks, and are decoded back to 30,000-dimensional vocabulary space using a basic affine transformation layer.

Section 11

the feed-forward size and filter size are synonymous. Both of them denote the number of dimensions in the middle layer of the feed-forward network. the hidden size and embedding size are synonymous. Both of them denote the number of real numbers used to represent a token. The notation for encoder stack is written as L/H. For example, BERTBASE is written as 12L/768H, BERTLARGE as 24L/1024H, and BERTTINY as 2L/128H.